

sse_challenge_2_Surname1Surname2

April 2, 2026

1 UE “Embedded Systems Security” (2025-2026) – Part 2

1.1 Introduction

This is the second challenge of the evaluated TP.

The **Context** section presents in detail the challenge. Several **questions** will guide you through the solution of the challenge.

The solution to this challenge will provide you the password to access to the second challenge.

Read carefully each section. Do not hesitate to contact the supervisor should you have any doubt or question.

1.1.1 What Material You Can Use

You can use: 1. All the code you developed during the TPs; 2. You can refer to the slides presented during the CMs; 3. You can refer to any other source (textbooks, research papers, blog posts, stackoverflow, ...); 4. You may use LLMs/AI coding assistants only to help you with coding.

1.1.2 What to Deliver

To have the challenges evaluated, you have to deliver the Jupyter notebooks completed with your answers.

You **MUST** deliver the notebooks before the end of the last TP. Deliver the notebooks (saved in a compressed format, such as TAR, GZ, ZIP, RAR, ...) on the moodle.

The jupyter notebooks must be named as ‘sse_challenge_X_Surname1Surname2’, where: * X: it is the challenge number; * Surname1 and Surname2 are the surnames of the binome.

Rename your compressed archive as **sse_challenges_Surname1Surname2**.

If you are working alone, just put your surname.

1.2 Context

You are assessing the attack surface for a client providing a FPGA-as-a-Service platform. In the first part of the challenge, you have assessed the presence of a vulnerability in the authentication system based on the AuDe device and FIDO protocol.

Now that you have access to the cloud service, you have to assess what an attacker may do once infiltrated. In a FPGA-as-a-Service platform, several FPGAs are put at disposal of the clients.

Each client can use only a subset of the available FPGAs, and this subset is potentially shared with the other clients: o a given FPGA, one or more clients can deploy whatever application they prefer.

With the access you have gained, you decide to install a *TDC* (Time-to-Digital converter, a circuit acting like an oscilloscope) in one of the FPGAs you have access and you recover some of the power traces.

Let us see what we are spying on: plot one trace from the set that you have recovered.

```
[ ]: import matplotlib.pyplot as plt

# Plot one trace
```

From the trace, you easily recognise that typical patterns of a well-known cryptosystem: the AES-128. This seems a rather easy target ...

1.2.1 Question 1:

Identify the *asset* that an attacker may try to recover from this cryptosystem.

Your Answer

```
[ ]:
```

1.2.2 Question 2:

What methodology/attack an attacker may exploit to recover the asset that you have identified? Provide a description (for instance, a pseudo-code) of the methodology.

Your Answer

```
[ ]:
```

1.2.3 Question 3:

Now that you have identified the asset and the methodology, you decide to recover such an asset.

Run the attack and display the result for the first element/byte of the asset. Explain what does the plot represent and the result of the analysis.

You may use the code below to run the attack; just complete the `aes_internal` function and indicate the `byte` you want to test.

```
[ ]: import numpy as np

sbox = [
    # 0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
    ↪
```

↪ 0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76, ↪
 ↪ # 0

↪ 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0, ↪
 ↪ # 1

↪ 0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15, ↪
 ↪ # 2

↪ 0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75, ↪
 ↪ # 3

↪ 0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84, ↪
 ↪ # 4

↪ 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf, ↪
 ↪ # 5

↪ 0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8, ↪
 ↪ # 6

↪ 0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2, ↪
 ↪ # 7

↪ 0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73, ↪
 ↪ # 8

↪ 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb, ↪
 ↪ # 9

↪ 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79, ↪
 ↪ # a

↪ 0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08, ↪
 ↪ # b

↪ 0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a, ↪
 ↪ # c

↪ 0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e, ↪
 ↪ # d

↪ 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf, ↪
 ↪ # e

```

    ↪ 0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16
    ↪ # f
]

def aes_internal(inputdata, key):
    return ???

HW = [bin(n).count("1") for n in range(0, 256)]

def mean(X):
    return np.sum(X, axis=0)/len(X)

def std_dev(X, X_bar):
    return np.sqrt(np.sum((X-X_bar)**2, axis=0))

def cov(X, X_bar, Y, Y_bar):
    return np.sum((X-X_bar)*(Y-Y_bar), axis=0)

```

```

[ ]: from tqdm.notebook import trange
maxcpa = [0] * 256

byte = ???

t_bar = mean(trace_array)
o_t = std_dev(trace_array, t_bar)

for kguess in trange(0, 256):
    hws = np.array([[HW[aes_internal(textin[byte],kguess)] for textin in
    ↪textin_array]]).transpose()

    hws_bar = mean(hws)
    o_hws = std_dev(hws, hws_bar)
    correlation = cov(trace_array, t_bar, hws, hws_bar)
    cpaoutput = correlation/(o_t*o_hws)
    maxcpa[kguess] = max(abs(cpaoutput))

guess = np.argmax(maxcpa)
guess_corr = max(maxcpa)

```

1.2.4 Your Answer

```

[ ]: # Plot here

```

1.2.5 Question 4:

It is likely you were not able to recover the asset.

Assuming the implementation of the attack is correct, you decide to better understand the set of traces that you have recovered.

Plot 5 traces and describe what you observe. Do you have any clue on why the attack did not work?

Your Answer

[]:

1.2.6 Question 5:

Ah right! The TDC introduce some *jitter* in the traces.

We have to remove such jitter. For doing so, one strategy is to: 1. Identify a pattern representing the beginning of the cryptosystem encryption; 2. For each trace, identify the beginning of the encryption; 3. For each trace, discard all that happens before the encryption;

Implement this *preprocessing* strategy.

Your Answer

[]:

1.2.7 Question 6

Now, run the attack on the preprocessed traces.

Try to recover the whole asset, and display the results of the attack methodology you ran.

Your Answer

[]:

1.3 Question 7

To check whether you recovered the whole asset, try to decrypt one of the ciphertexts associated with the challenge and see if it matches the related plaintext.

```
[4]: import numpy as np
      from aes import decrypt
      np.set_printoptions(formatter={'int':lambda x:hex(int(x))})
      plaintexts = np.load('jittery-aes-plaintexts.npy')
      ciphertext = np.load('jittery-aes-ciphertexts.npy')[0, :]
      key = # your key

      assert np.all(np.frombuffer(decrypt(key, ciphertext), dtype = np.uint8) ==
      ↪plaintexts[0, :])
```

Have you recovered the correct key?

1.3.1 Your Answer

[]:

1.4 End of the Challenge

Save this notebook, rename it as `sse_challenge_2_Surname1Surname2`, where Surname1 and Surname2 are the surnames of the pair that has completed this challenge.